

Benchmark & Pipeline ML

NYC Taxi: Profiling com Pandas

Experimento 5 - Ciência de Dados em Larga Escala

Autores: **Renata Azevedo, Giuliano e Ni**

Objetivo

Machine Learning

Desenvolver um pipeline completo em contexto de Big Data,

Escopo da Análise

Análise de desempenho de diferentes bibliotecas Python

Dados: Arquivos **.parquet** de viagens NYC Taxi (Ano 2026).

Nota: também testamos Cprofile para capturar o comportamento determinístico das chamadas de função.

Ferramentas usados

- Pandas: Principal biblioteca para análise de dados
- Dask: computação paralela e distribuída,
- Spark/Koalas: processamento distribuído em memória.
- Modin:: utilização pandas multi-cpus
- Joblib: biblioteca de pararelização
- Rapids: Utilização de GPU para tarefas



Tarefas

Task 1 — Leitura dos Dados

Primeiro experimento: Contar quantas viagens ocorreram por tipo de pagamento.

Task 2 — Agregações e Filtragem

Segundo experimento: GroupBy. Calcular o valor médio da tarifa para cada tipo de fornecedor.

Terceiro experimento: Filtrar apenas as viagens que tiveram mais de 2 passageiros e calcular a distância média percorrida para esse grupo específico.

Task 3 — Join de Tabelas

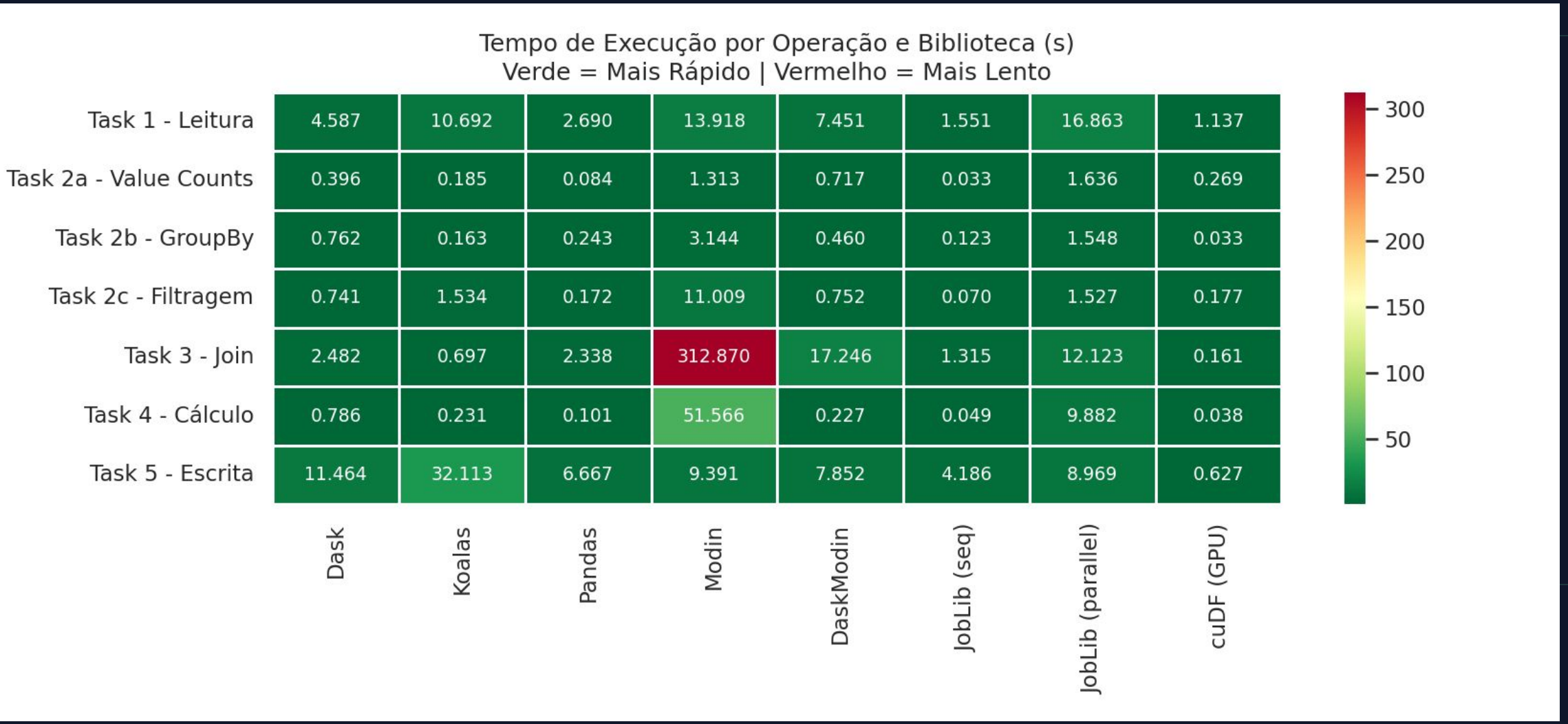
traduzir IDs em nomes de localização.

Task 4 — Cálculo Aritmético

gorjeta por passageiro (`tip_per_passenger = tip_amount / passenger_count`).

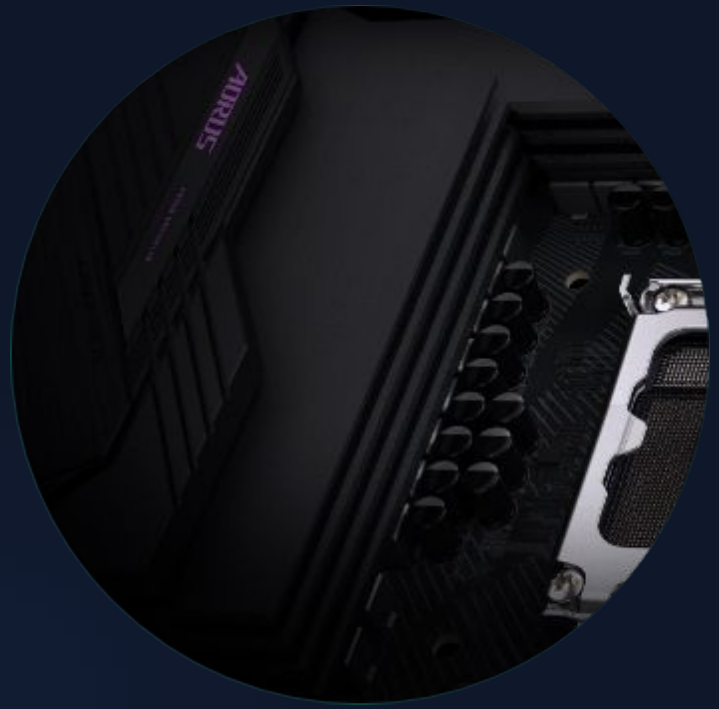
Task 5 — Escrita dos Dados

Resultado das tarefas



Vencedor
cuDF (GPU)
JobLib (seq)
cuDF (GPU)
JobLib (seq)
cuDF (GPU)
cuDF (GPU)
cuDF (GPU)

Resumo da analise



- Para utilização geral e simplicidade de datasets, o Pandas continua a ser uma escolha sólida.
- Bibliotecas como Dask, Modin e Joblib podem oferecer vantagens específicas dependendo do ambiente e do tipo de workload, por exemplo para dados de grande dimensão.
- RAPIDS/cuDF foi claramente a solução mais eficiente, dominando todas as tarefas avaliadas e apresentando acelerações substanciais relativamente às alternativas baseadas em CPU.
- Os resultados confirmam o potencial da computação acelerada por GPU para aplicações de Big Data e análise de dados em larga escala.

ML

Tarefa ML:

Prever fare_amount (tarifa da viagem)

Modelo	Tipo	Objetivo
XGBRegressor	Regressão	Prever `fare_amount` como variável contínua
LogisticRegression	Classificação	Prever `fare_amount` discretizado em 4 classes

Escopo da Análise

Análise de desempenho, usando Biblioteca Panda como referência base e Cudf+Cuml para utilização de GPU para ML

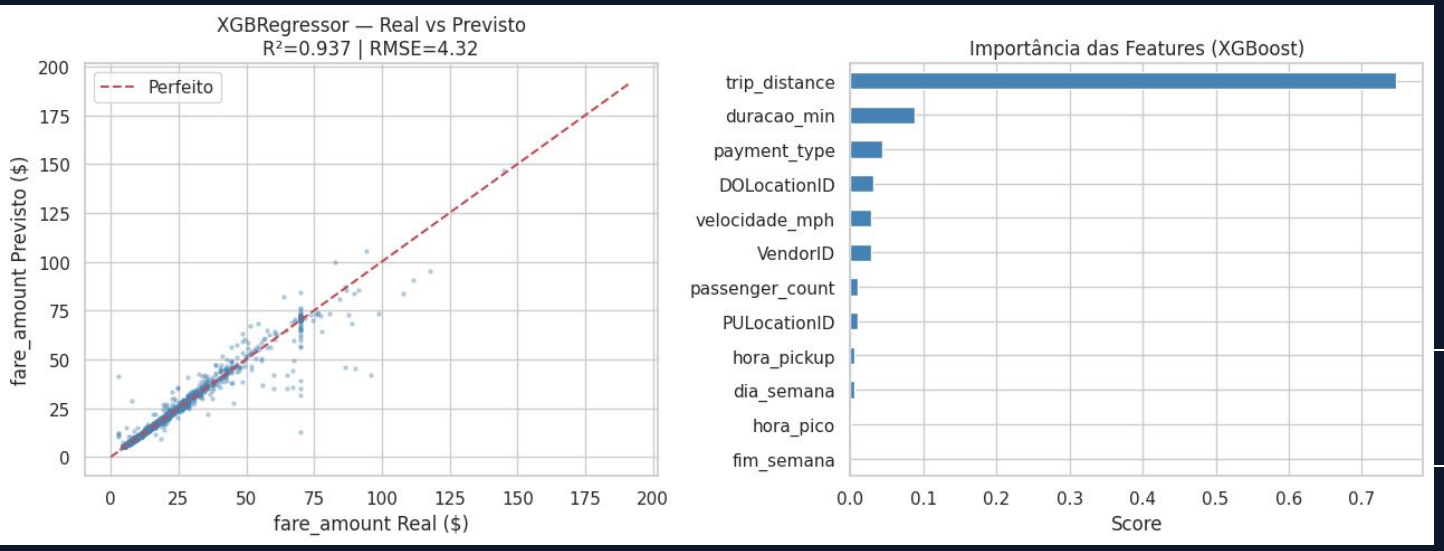
Dados: Arquivos **.parquet** de viagens NYC Taxi (Ano 2026).

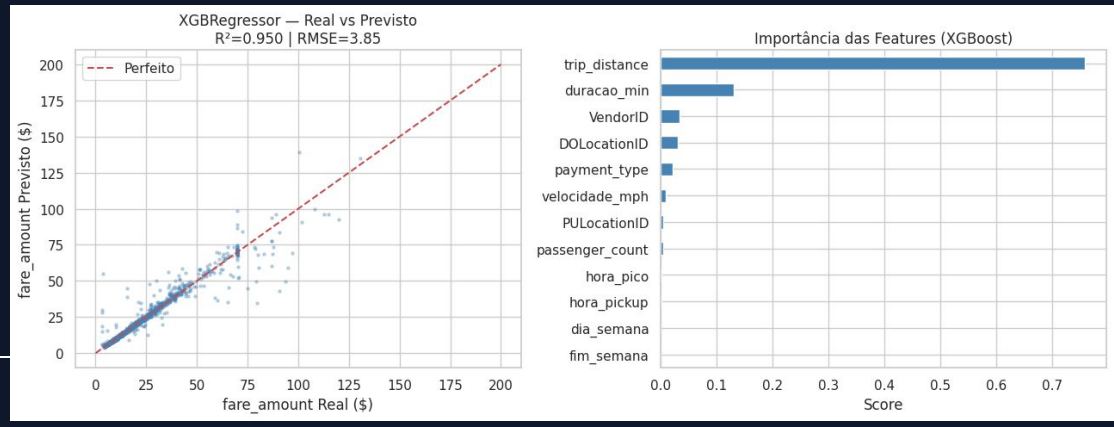
dataset reduzido (amostra de ~40 000 registos) para garantir execução rápida e evitar estouro de memória

Amostra de 4.000.000 registos para comparação entre Panda e GPU

Resultados ML Panda

Amostra 40000 vs 4000000





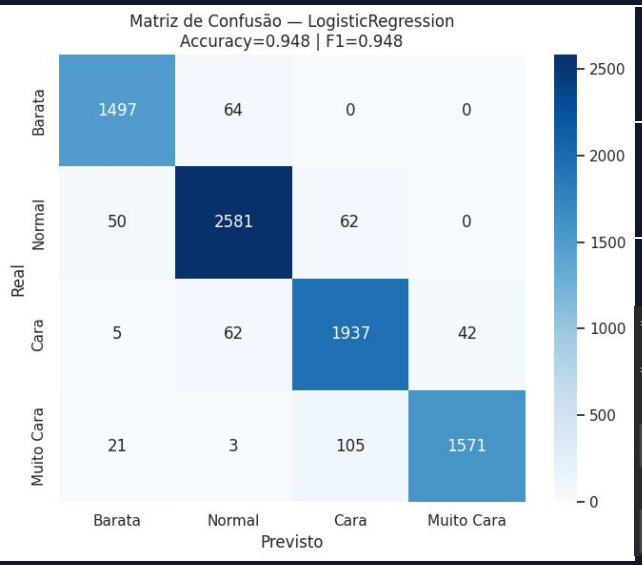
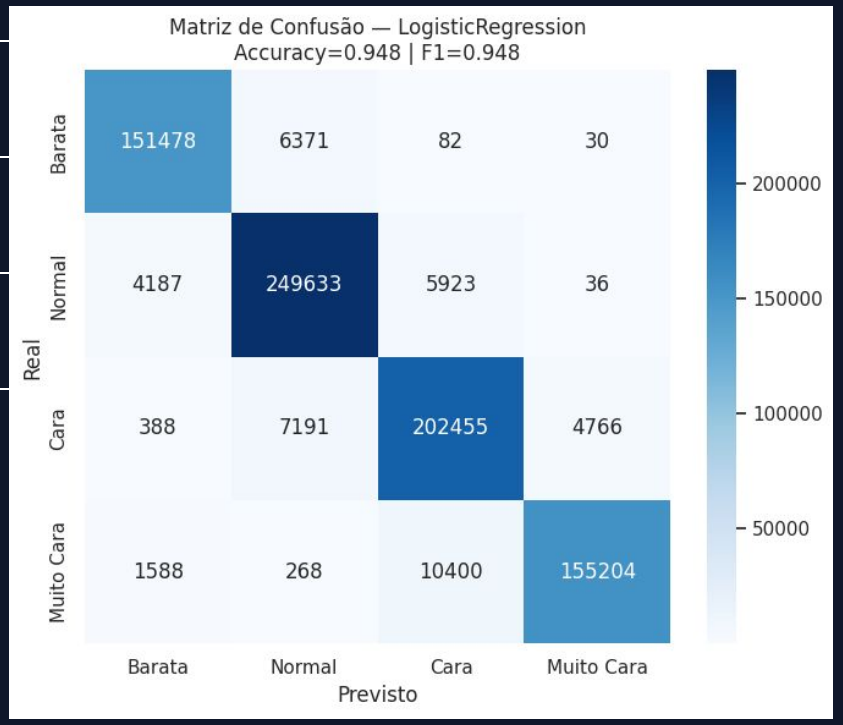


TABELA FINAL — MÉTRICAS DE MACHINE LEARNING

	Modelo	Tipo	RMSE	MAE	R²	Accuracy	F1	Tempo treino (s)
0	XGBRegressor	Regressão	4.3196	1.2865	0.9369	NaN	NaN	4.66
1	LogisticRegression	Classificação	NaN	NaN	NaN	0.9483	0.9483	4.77
2	LogisticRegression (Iris)	Classificação	NaN	NaN	NaN	0.9333	0.9333	0.27

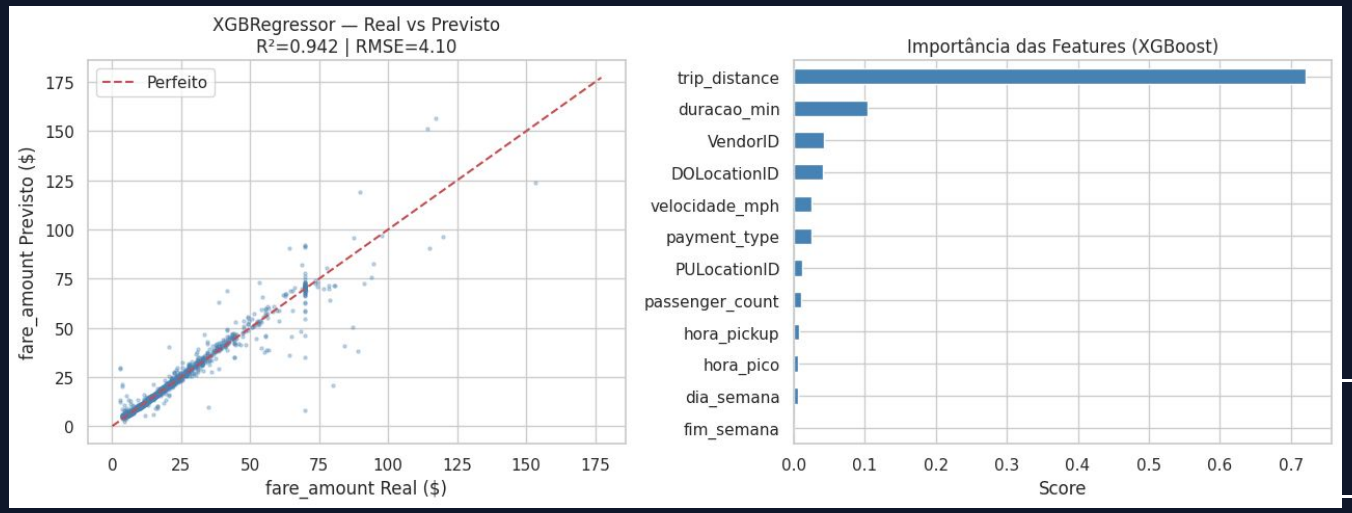
TABELA FINAL — MÉTRICAS DE MACHINE LEARNING

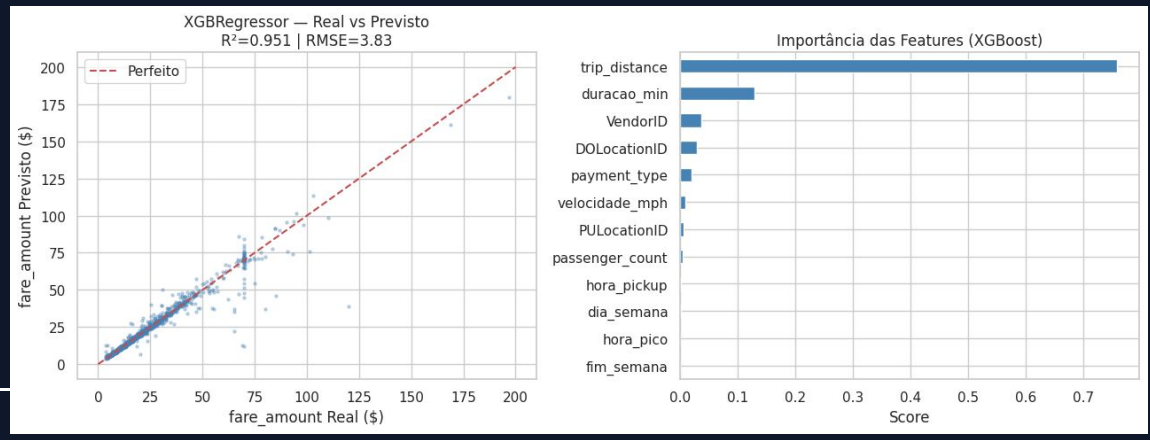
	Modelo	Tipo	RMSE	MAE	R²	Accuracy	F1	Tempo treino (s)
0	XGBRegressor	Regressão	3.8475	1.1372	0.9504	NaN	NaN	387.76
1	LogisticRegression	Classificação	NaN	NaN	NaN	0.9485	0.9485	493.64
2	LogisticRegression (Iris)	Classificação	NaN	NaN	NaN	0.9333	0.9333	0.05



Resultados ML Cudf/Cuml

Amostra 40000 vs 4000000





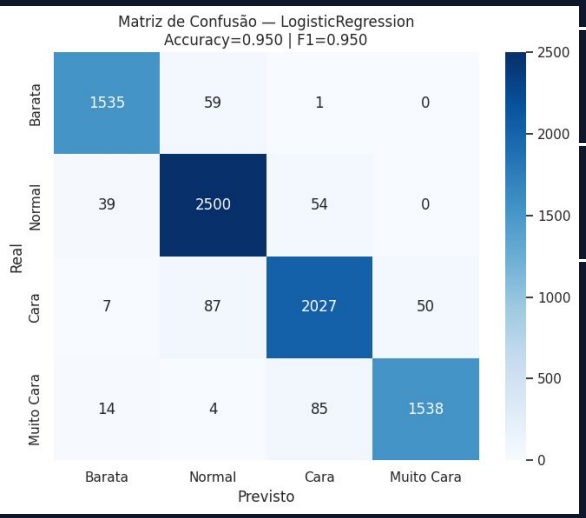
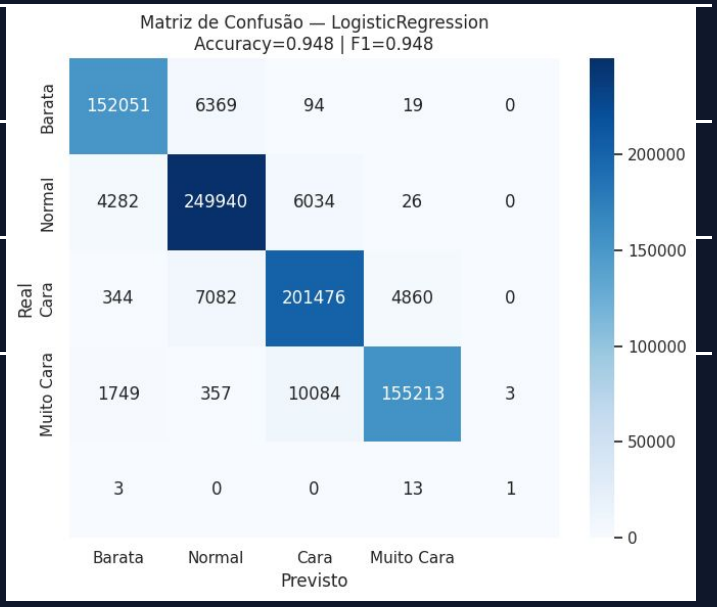


TABELA FINAL — MÉTRICAS DE MACHINE LEARNING

	Modelo	Tipo	RMSE	MAE	R²	Accuracy	F1	Tempo treino (s)
1	LogisticRegression (GPU cuML)	Classificação	NaN	NaN	NaN	0.9500	0.9500	0.33
2	LogisticRegression (Iris GPU cuML)	Classificação	NaN	NaN	NaN	0.9333	0.9333	0.01
0	XGBRegressor (GPU)	Regressão	4.0955	1.2789	0.9417	NaN	NaN	3.26

TABELA FINAL — MÉTRICAS DE MACHINE LEARNING

	Modelo	Tipo	RMSE	MAE	R²	Accuracy	F1	Tempo treino (s)
1	LogisticRegression (GPU cuML)	Classificação	NaN	NaN	NaN	0.9484	0.9484	30.04
2	LogisticRegression (Iris GPU cuML)	Classificação	NaN	NaN	NaN	0.9333	0.9333	0.02
0	XGBRegressor (GPU)	Regressão	3.8332	1.1389	0.9507	NaN	NaN	70.66



Conclusão

Pipeline GPU — Principais Conclusões:

- A GPU (RAPIDS/cuML) não melhora a precisão dos modelos, mas sim a escalabilidade

- O ganho principal é no tempo de treino, especialmente em datasets grandes Em CPU, o pipeline torna-se rapidamente impraticável à escala maior (ex: >500s vs ~90s)

- A arquitetura GPU permite manter performance estável mesmo com aumento de dados

